

# RentalPro

## Digital Rental Management System

### DELIVERABLE 4

#### Complete Project Report

| Field              | Details                                                                                                                          |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Student Name       | Abdur-Rahmaan Muhammad Ali                                                                                                       |
| Student ID         | ST10201821                                                                                                                       |
| Course Code        | IT401                                                                                                                            |
| Project Name       | RentalPro — Digital Rental Management System                                                                                     |
| Deliverable        | Deliverable 4 — Complete Project Report                                                                                          |
| Version            | v1.0                                                                                                                             |
| Chapters           | Introduction · Object-Oriented Analysis · Object-Oriented Design · Database Design · Software Development · Testing · Conclusion |
| Appendices         | Appendix A — Installation Guide · Appendix B — User Manual · Appendix C — Updates Since Deliverable 4                            |
| Prior Deliverables | Deliverable 1 — SRS · Deliverable 2 — UML & Database Design · Deliverable 3 — Partially-Developed Application                    |

# Table of Contents

|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>Chapter 1: Introduction.....</b>                                                   | <b>5</b>  |
| 1.1 Background.....                                                                   | 5         |
| 1.2 Problem Statement.....                                                            | 5         |
| 1.3 Objectives.....                                                                   | 5         |
| 1.4 Scope.....                                                                        | 6         |
| 1.5 Limitations.....                                                                  | 6         |
| 1.6 Project Overview.....                                                             | 6         |
| <b>Chapter 2: Object-Oriented Analysis.....</b>                                       | <b>8</b>  |
| 2.1 Overview.....                                                                     | 8         |
| 2.2 System Actors.....                                                                | 8         |
| 2.3 Use Case Model.....                                                               | 8         |
| 2.3.1 Detailed Use Case — UC-09: Upload Lease Document.....                           | 9         |
| 2.3.2 Detailed Use Case — UC-13: Pay Rent Online.....                                 | 10        |
| 2.3.3 Detailed Use Case — UC-18/UC-19/UC-20/UC-21: Maintenance Request Lifecycle..... | 11        |
| 2.4 Non-Functional Requirements Analysis.....                                         | 11        |
| 2.5 Domain (Conceptual Class) Analysis.....                                           | 12        |
| <b>Chapter 3: Object-Oriented Design.....</b>                                         | <b>13</b> |
| 3.1 Architectural Style.....                                                          | 13        |
| 3.2 Design Patterns Used.....                                                         | 13        |
| 3.3 Package Structure.....                                                            | 14        |
| 3.4 Class Diagram.....                                                                | 15        |
| 3.4.1 Class Relationships and Multiplicity.....                                       | 15        |
| 3.5 Interaction Design — Sequence of Key Operations.....                              | 16        |
| 3.5.1 Sequence — Pay Rent Online (UC-13).....                                         | 16        |
| 3.5.2 Sequence — Submit & Assign Maintenance Request (UC-18 & UC-19).....             | 17        |
| 3.6 Activity Design — Maintenance Request Lifecycle.....                              | 17        |
| 3.7 Design Decisions and Rationale.....                                               | 18        |
| <b>Chapter 4: Database Design.....</b>                                                | <b>19</b> |
| 4.1 Database Technology.....                                                          | 19        |
| 4.2 Entity Relationship Diagram.....                                                  | 19        |
| 4.3 Schema Design Decisions.....                                                      | 19        |
| 4.4 Entity Summary and Cardinalities.....                                             | 20        |
| 4.5 Table Definitions.....                                                            | 21        |
| 4.5.1 users.....                                                                      | 21        |
| 4.5.2 properties.....                                                                 | 21        |
| 4.5.3 units.....                                                                      | 22        |
| 4.5.4 tenants.....                                                                    | 22        |
| 4.5.5 leases.....                                                                     | 22        |
| 4.5.6 rent_payments, maintenance_requests, and remaining tables.....                  | 23        |
| 4.6 Indexes.....                                                                      | 23        |
| 4.7 Referential Integrity.....                                                        | 23        |

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Chapter 5: Software Development.....</b>              | <b>25</b> |
| 5.1 Technology Stack.....                                | 25        |
| 5.2 Development Methodology.....                         | 25        |
| 5.3 Application Factory and Configuration.....           | 25        |
| 5.4 Route-to-Use-Case Mapping.....                       | 26        |
| 5.5 Key Algorithms.....                                  | 27        |
| 5.5.1 Recurring Rent Due-Date Calculation.....           | 27        |
| 5.5.2 Role-Based Access Control and IDOR Protection..... | 27        |
| 5.5.3 Idempotent Numbering and Payment Recording.....    | 28        |
| 5.5.4 Dev-Mode Fallback for External Services.....       | 28        |
| 5.6 Security Implementation.....                         | 28        |
| 5.7 Deployment.....                                      | 29        |
| 5.7.1 Docker.....                                        | 29        |
| 5.7.2 Continuous Integration.....                        | 29        |
| 5.8 Implementation Defects Found and Fixed.....          | 29        |
| <b>Chapter 6: Testing.....</b>                           | <b>31</b> |
| 6.1 Testing Strategy.....                                | 31        |
| 6.2 Test Environment.....                                | 31        |
| 6.3 Test Suite Structure.....                            | 31        |
| 6.4 Representative Test Cases.....                       | 32        |
| 6.5 Test Results.....                                    | 33        |
| 6.6 Known Test Limitations.....                          | 33        |
| 6.7 How to Run the Test Suite.....                       | 33        |
| <b>Chapter 7: Conclusion.....</b>                        | <b>35</b> |
| 7.1 Summary of Achievements.....                         | 35        |
| 7.2 Challenges Encountered.....                          | 35        |
| 7.3 Limitations.....                                     | 35        |
| 7.4 Future Work.....                                     | 35        |
| 7.5 Final Remarks.....                                   | 36        |
| <b>Appendix A: Installation Guide.....</b>               | <b>37</b> |
| A.1 Prerequisites.....                                   | 37        |
| A.2 Option 1 — Docker (Recommended).....                 | 37        |
| A.3 Option 2 — Local Installation (Without Docker).....  | 37        |
| A.4 Environment Variables.....                           | 38        |
| A.5 Demo Credentials.....                                | 38        |
| A.6 Running the Test Suite.....                          | 38        |
| A.7 Troubleshooting.....                                 | 39        |
| <b>Appendix B: User Manual.....</b>                      | <b>40</b> |
| B.1 Getting Started.....                                 | 40        |
| B.2 Property Owner & Property Manager Guide.....         | 40        |
| B.2.1 Add a Property.....                                | 40        |
| B.2.2 Add a Unit.....                                    | 40        |
| B.2.3 Onboard a Tenant.....                              | 40        |

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| B.2.4 Upload a Lease and Activate a Tenancy.....                   | 40        |
| B.2.5 Record a Rent Payment.....                                   | 41        |
| B.2.6 Generate a Rent Statement.....                               | 41        |
| B.2.7 Manage a Maintenance Request.....                            | 41        |
| B.2.8 View Reports.....                                            | 41        |
| B.3 Tenant Guide.....                                              | 41        |
| B.3.1 Your Dashboard.....                                          | 41        |
| B.3.2 Pay Rent Online.....                                         | 41        |
| B.3.3 View Your Payment History.....                               | 41        |
| B.3.4 Submit a Maintenance Request.....                            | 42        |
| B.3.5 Respond to a Completed Request.....                          | 42        |
| B.4 Maintenance Staff Guide.....                                   | 42        |
| B.5 Common Questions.....                                          | 42        |
| <b>Appendix C: Updates Since Deliverable 4 (October 2026).....</b> | <b>43</b> |
| C.1 Summary of Changes.....                                        | 43        |
| C.2 Current Figures.....                                           | 44        |
| C.3 Language Support and the SRS.....                              | 44        |
| C.4 The Current Interface.....                                     | 45        |
| C.5 Updated Demo Accounts and Data.....                            | 47        |

# Chapter 1: Introduction

*Note: this document records RentalPro as it stood at Deliverable 4, and every figure in it (93 tests, 77% coverage, 31 templates) is that of the system at the time. Appendix C summarises everything added since — the eleven gaps closed in Deliverable 5, the billing engine, the redesigned Harare interface, English, Shona and Arabic, production hardening and deployment to Render — with today's figures (18 tables, 305 tests, 86% coverage).*

## 1.1 Background

Small and mid-sized property owners and managers commonly track tenants, rent payments, and maintenance work through a mixture of spreadsheets, paper files, and ad-hoc messaging. As a portfolio grows past a handful of units, this manual approach becomes error-prone: rent due dates are missed, maintenance requests are lost between a phone call and a follow-up, and there is no single place to see which units are occupied, which tenants are behind on rent, or how much a property earned last month.

RentalPro was conceived to replace that fragmented workflow with a single, role-aware web application that a Property Owner, a Property Manager, a Tenant, and a Maintenance Staff member can all use for their part of the rental lifecycle — from listing a property, to onboarding a tenant and activating a lease, to collecting rent and resolving a maintenance ticket.

## 1.2 Problem Statement

Property owners operating small-to-mid-sized rental portfolios (1–500 units) lack an affordable, purpose-built system that unifies four things that are currently handled separately: property and unit records, tenant and lease records, rent collection and arrears tracking, and maintenance request handling. The absence of a unified system causes:

- Missed or late rent reminders because no system tracks due dates automatically.
- No single source of truth for who owes what, across how many properties.
- Maintenance requests that are reported verbally and easily forgotten or duplicated.
- No audit trail of who changed what, which matters when a payment or a lease term is disputed.
- No role separation — anyone with access to the spreadsheet can see and edit everything, regardless of whether they are the owner, a manager, or a tenant.

## 1.3 Objectives

**General objective:** Design, build, and test a web-based Rental Management System that digitises the full rental lifecycle — property listing, tenant records, rent tracking, and maintenance requests — for four operational user roles plus an administrator.

**Specific objectives:**

- Analyse the domain and produce a Software Requirements Specification covering functional and non-functional requirements (Deliverable 1).
- Design the system's object-oriented structure and relational database schema (Deliverable 2).
- Implement a working application covering all 25 identified use cases across 5 user roles (Deliverable 3).

- Verify the implementation with an automated test suite covering business logic, authentication/authorisation, end-to-end feature flows, scheduled jobs, and input validation.
- Document the finished system in a complete project report covering the analysis, design, database, development, and testing performed, plus installation and user documentation (this document, Deliverable 4).

## 1.4 Scope

RentalPro covers four functional modules, consistent with the scope defined in the Software Requirements Specification (Deliverable 1):

| Module               | Description                                                           |
|----------------------|-----------------------------------------------------------------------|
| Property Listing     | Register properties, manage unit details, track occupancy status      |
| Tenant Records       | Onboard tenants, upload lease documents, maintain contact and history |
| Rent Tracking        | Record rent payments, generate due-date alerts, produce statements    |
| Maintenance Requests | Submit, assign, track, and close maintenance work orders              |

The system targets small-to-mid-sized property management operations (1 to 500 units) and is not a public property marketplace, nor does it handle real-estate transactions or mortgage processing.

## 1.5 Limitations

This report documents the system as actually implemented and tested, including known simplifications made against the original design in Deliverable 2, in the interest of a working, honestly-assessed deliverable rather than an aspirational one:

- File storage is local disk in development; the design supports swapping in AWS S3 for production but this has not been implemented or tested.
- PDF generation uses reportlab rather than the originally specified WeasyPrint, to avoid native system dependencies in the Docker image — the generated documents (rent statements, property reports) are equivalent in content.
- Payment, email, and SMS integrations (Stripe, SendGrid, Twilio) run in a dev-mode fallback unless real API keys are supplied; this was a deliberate choice so the whole system, including its automated test suite, runs fully offline.
- Multi-currency and multi-language (Arabic/English) support described in the SRS is represented in the data model (currency per property) but a translated UI was not implemented.
- No formal database migration history (Alembic) has been generated; schema changes during development were applied via SQLAlchemy's `create_all()`.

## 1.6 Project Overview

RentalPro is a server-rendered Flask web application backed by a relational database (SQLite in development and test, PostgreSQL in Docker/production). It supports five roles — Property Owner, Property Manager, Tenant, Maintenance Staff, and Administrator — each with a role-specific dashboard

and a restricted set of actions enforced both by route-level guards and by data-level ownership checks. The remaining chapters of this report cover the analysis (Chapter 2), design (Chapters 3–4), implementation (Chapter 5), and verification (Chapter 6) of the system, followed by a conclusion (Chapter 7) and three appendices covering installation, day-to-day use, and what has changed since Deliverable 4.

## Chapter 2: Object-Oriented Analysis

### 2.1 Overview

This chapter analyses the RentalPro domain in object-oriented terms: who the system's actors are, what they need to accomplish (its use cases), and what real-world concepts (candidate objects) the system must model. It condenses and confirms the analysis first carried out in the Software Requirements Specification (Deliverable 1), cross-checked against what was actually implemented and tested.

### 2.2 System Actors

RentalPro recognises five actors. Four are human roles with a dashboard and permitted actions; the fifth (System/Scheduler) is an automated actor that triggers time-based use cases without a human present.

| Actor              | Description                                                                                                            | Primary Use Cases                 |
|--------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Property Owner     | Ultimate account holder; owns one or more properties; full management access, exclusive right to create new properties | UC-03 to UC-06, UC-24, UC-25      |
| Property Manager   | Day-to-day operational user acting on the owner's behalf; same access as Owner except cannot create new properties     | UC-04 to UC-22                    |
| Tenant             | Renter occupying a unit; self-service portal only — own lease, own payments, own maintenance requests                  | UC-10, UC-13, UC-17, UC-18, UC-21 |
| Maintenance Staff  | Technician who executes assigned work orders; queue-only access                                                        | UC-20                             |
| System / Scheduler | Automated actor; runs the four background jobs on a daily schedule                                                     | UC-11, UC-14, UC-15, UC-23        |

An Administrator role also exists at the data-access level (bypasses ownership checks everywhere) but has no use cases of its own in the current implementation beyond the management actions already listed for Owner/Manager.

### 2.3 Use Case Model

Twenty-five use cases were identified across the four functional modules plus authentication and reporting. Every use case listed below has a corresponding route (or scheduled job) in the implementation and at least one automated test — see Chapter 6 for the verification evidence and Section 5.4 for the exact route mapping.

| UC    | Name                                   | Primary Actor | Module         |
|-------|----------------------------------------|---------------|----------------|
| UC-01 | Register Account / Login               | All Users     | Authentication |
| UC-02 | Manage User Roles & Permissions (RBAC) | System Admin  | Authentication |

| UC    | Name                                 | Primary Actor     | Module           |
|-------|--------------------------------------|-------------------|------------------|
| UC-03 | Add New Property                     | Property Owner    | Property Listing |
| UC-04 | Add / Edit Unit                      | Owner / Manager   | Property Listing |
| UC-05 | View Property Dashboard              | Owner / Manager   | Property Listing |
| UC-06 | Archive Property / Unit              | Owner / Manager   | Property Listing |
| UC-07 | Create Tenant Profile                | Property Manager  | Tenant Records   |
| UC-08 | Invite Tenant via Link               | Property Manager  | Tenant Records   |
| UC-09 | Upload Lease Document                | Property Manager  | Tenant Records   |
| UC-10 | View Lease and Profile               | Tenant            | Tenant Records   |
| UC-11 | Send Lease Expiry Alert              | System            | Tenant Records   |
| UC-12 | Record Rent Payment                  | Property Manager  | Rent Tracking    |
| UC-13 | Pay Rent Online                      | Tenant            | Rent Tracking    |
| UC-14 | Send Rent Due Alert                  | System            | Rent Tracking    |
| UC-15 | Send Overdue Notification            | System            | Rent Tracking    |
| UC-16 | Generate Rent Statement              | Property Manager  | Rent Tracking    |
| UC-17 | View Rent History                    | Tenant / Manager  | Rent Tracking    |
| UC-18 | Submit Maintenance Request           | Tenant            | Maintenance      |
| UC-19 | Assign Maintenance Request           | Property Manager  | Maintenance      |
| UC-20 | Update Request Status                | Maintenance Staff | Maintenance      |
| UC-21 | Confirm / Reopen Request             | Tenant            | Maintenance      |
| UC-22 | Log Maintenance Cost                 | Property Manager  | Maintenance      |
| UC-23 | Escalate Overdue Request             | System            | Maintenance      |
| UC-24 | Generate Property Performance Report | Owner / Manager   | Reporting        |
| UC-25 | Export Report (PDF / JSON)           | Owner / Manager   | Reporting        |

### 2.3.1 Detailed Use Case — UC-09: Upload Lease Document

| Field         | Description                                                                         |
|---------------|-------------------------------------------------------------------------------------|
| Actors        | Property Manager (primary)                                                          |
| Preconditions | Manager is logged in; tenant profile exists; a signed lease PDF is ready for upload |
| Trigger       | Manager clicks "Upload Lease Document" from a                                       |

| Field          | Description                                                                                                                                                                                                                                                                                                                                          |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                | tenant's profile                                                                                                                                                                                                                                                                                                                                     |
| Main Flow      | 1. Manager selects a vacant unit. 2. Manager enters lease dates, monthly rent, deposit, and due day. 3. Manager uploads the signed PDF. 4. System validates the file is a genuine PDF (magic-byte check) and the dates are consistent. 5. System creates the Lease record and sets the unit to Occupied. 6. System emails the tenant a confirmation. |
| Exception Flow | Non-PDF or oversized file is rejected before any record is created; a unit already leased to a different active tenant blocks the upload.                                                                                                                                                                                                            |
| Postconditions | Lease is ACTIVE; unit status is OCCUPIED; lease appears in the tenant's rent history going forward.                                                                                                                                                                                                                                                  |
| Verified by    | tests/test_routes.py —<br>test_upload_lease_sets_unit_occupied,<br>test_upload_lease_rejects_non_pdf                                                                                                                                                                                                                                                 |

### 2.3.2 Detailed Use Case — UC-13: Pay Rent Online

| Field          | Description                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Actors         | Tenant (primary), Payment Gateway (secondary)                                                                                                                                                                                                                                                                                                                                                                                       |
| Preconditions  | Tenant is logged in with an active lease and an outstanding balance                                                                                                                                                                                                                                                                                                                                                                 |
| Trigger        | Tenant clicks "Pay Now" in the portal                                                                                                                                                                                                                                                                                                                                                                                               |
| Main Flow      | 1. System shows the current balance. 2. Tenant confirms payment. 3. System creates a checkout session with the payment gateway (or a simulated dev-mode session when no live key is configured). 4. Gateway confirms success via redirect/webhook. 5. System records the payment idempotently against the gateway reference and recalculates the balance. 6. System emails a receipt to the tenant and notifies the property owner. |
| Exception Flow | A retried webhook with the same gateway reference does not create a second payment (idempotency); a tampered or replayed webhook signature is rejected.                                                                                                                                                                                                                                                                             |
| Postconditions | Balance is reduced by the payment amount; a receipt number is issued; the payment is visible in the tenant's rent history.                                                                                                                                                                                                                                                                                                          |
| Verified by    | tests/test_routes.py —<br>test_pay_rent_online_dev_mode_end_to_end;<br>tests/test_validation.py — Stripe signature and idempotency tests                                                                                                                                                                                                                                                                                            |

### 2.3.3 Detailed Use Case — UC-18/UC-19/UC-20/UC-21: Maintenance Request Lifecycle

These four use cases form one continuous workflow and were analysed together because they share a single state machine on MaintenanceRequest: SUBMITTED → ASSIGNED → IN\_PROGRESS → COMPLETED → CLOSED, with a REOPEN transition back to SUBMITTED.

| Step | Actor             | Action                                                               | Resulting Status          |
|------|-------------------|----------------------------------------------------------------------|---------------------------|
| 1    | Tenant            | Submits a request with title, category, and severity                 | SUBMITTED                 |
| 2    | Property Manager  | Assigns the request to a Maintenance Staff member with a target date | ASSIGNED                  |
| 3    | Maintenance Staff | Marks work as started                                                | IN_PROGRESS               |
| 4    | Maintenance Staff | Marks work as completed, with notes                                  | COMPLETED                 |
| 5a   | Tenant            | Confirms resolution (optionally rates 1–5)                           | CLOSED                    |
| 5b   | Tenant            | Reopens with a reason instead                                        | SUBMITTED (cycle repeats) |

An Emergency-severity request additionally triggers an immediate SMS to the property owner at submission time, and the automated escalation job (UC-23) flags any request whose target date has passed.

## 2.4 Non-Functional Requirements Analysis

The functional analysis above is paired with the non-functional requirements from Deliverable 1, grouped into the categories the implementation addresses directly:

| Category    | Requirement (from SRS) → How it was analysed for design                                                                                                                                                                                                                     |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Security    | Passwords must be hashed, sessions must expire, RBAC must be enforced per role, and a user must never read another user's data — drove the design of a single role_required() decorator plus a separate per-resource assert_owner()/assert_tenant_self() check (Chapter 3). |
| Reliability | Payment records must be immutable and auditable — drove the append-only AuditLog table and the decision never to UPDATE or DELETE a RentPayment row (Chapter 4).                                                                                                            |
| Performance | Dashboard and listing pages must load quickly under normal load — drove the indexing strategy in Chapter 4 (composite indexes on the columns every dashboard query filters by).                                                                                             |
| Usability   | A manager must be able to add a property and unit without training; a tenant must be able to submit a request in minutes — drove the flat, role-scoped                                                                                                                      |

| Category    | Requirement (from SRS) → How it was analysed for design                                                                                                                               |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             | navigation described in Appendix B.                                                                                                                                                   |
| Scalability | The design should not need architectural change up to 100,000 units — drove the choice of UUID primary keys (safe under distributed/concurrent inserts) over auto-increment integers. |

## 2.5 Domain (Conceptual Class) Analysis

Reading the use cases and requirements for candidate nouns yields the following analysis-level classes, later refined into the 13 implementation classes documented in Chapter 3:

| Candidate Class    | Responsibility                                            |
|--------------------|-----------------------------------------------------------|
| User               | Authenticate; hold identity and role                      |
| Property           | Group units under one owner; report occupancy             |
| Unit               | The rentable thing; know its own status and current lease |
| Tenant             | Renter-specific profile data, linked one-to-one to a User |
| Lease              | The contract; know how much is owed as of any date        |
| RentPayment        | An immutable record of money received                     |
| MaintenanceRequest | A ticket; own its lifecycle state and cost total          |
| Notification       | A record that a message was (or should be) sent           |
| AuditLog           | An immutable record that something happened               |

Two further analysis classes — TenantInvitation and PasswordReset — emerged not from the SRS's nouns directly but from its stated business rules (a 48-hour invitation window, a time-limited password reset link) once those rules were modelled as first-class, expiring tokens rather than fields bolted onto User. This refinement step is carried into the design in Chapter 3.

# Chapter 3: Object-Oriented Design

## 3.1 Architectural Style

RentalPro is a server-rendered, layered web application: a Flask application factory (backend/\_\_\_init\_\_\_py) wires together a presentation layer of Jinja2 templates, an application layer of nine route Blueprints, a domain/service layer of pure-Python modules, and a data layer of thirteen SQLAlchemy models. This is a deliberate departure from the micro-service-oriented architecture sketched in Deliverable 2 (separate PropertyService, TenantService, etc. behind an API gateway): for a single-deployment application of this size, a layered monolith is simpler to build, test, and reason about, while still keeping each concern in its own module. Figure 1 shows the architecture as implemented.

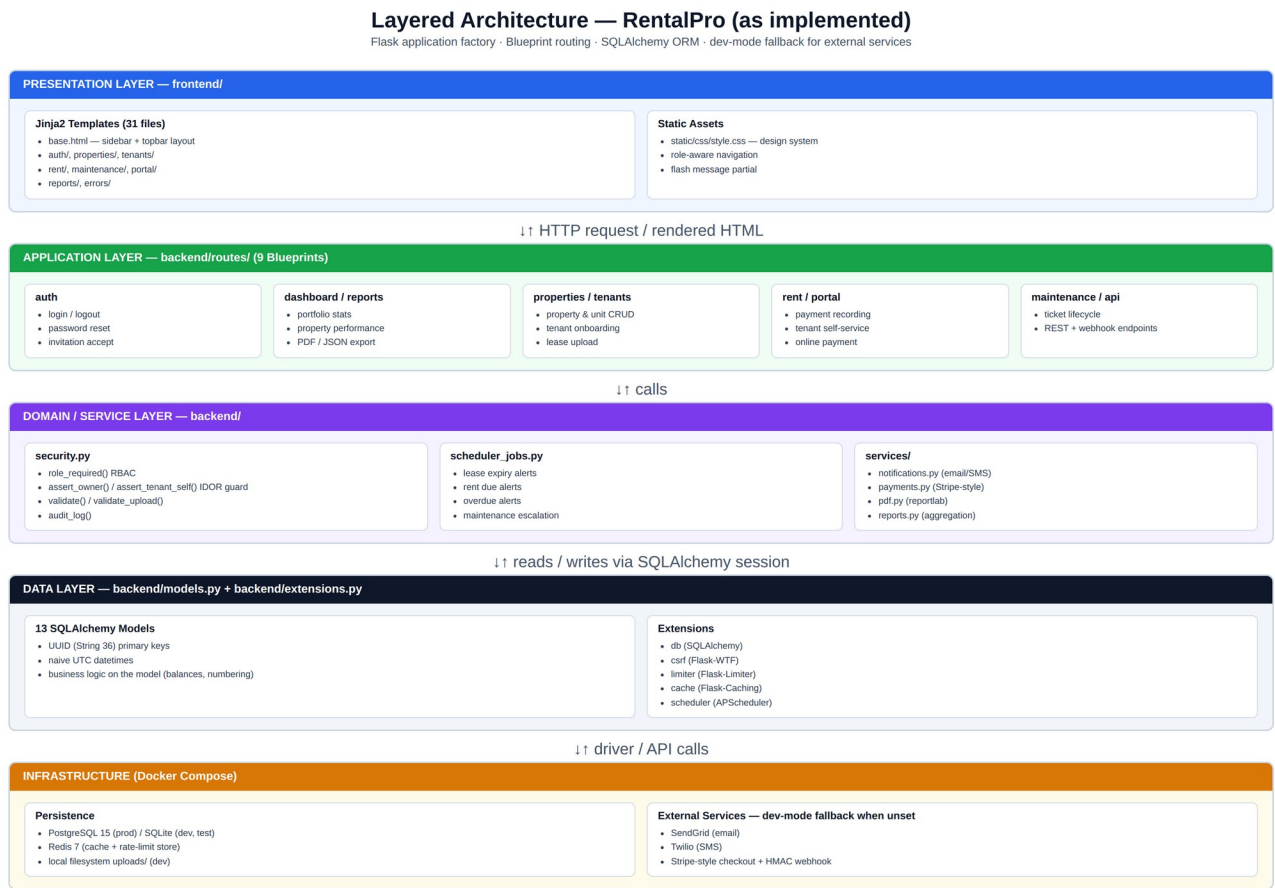


Figure 1: Layered Architecture — RentalPro (as implemented)

## 3.2 Design Patterns Used

| Pattern                                | Where                               | Why                                                                                              |
|----------------------------------------|-------------------------------------|--------------------------------------------------------------------------------------------------|
| Application Factory                    | backend/___init___py — create_app() | Allows a fresh app instance per test (TestConfig) without global state leaking between tests     |
| Blueprint (Front Controller / MVC-ish) | backend/routes/*.py — 9 blueprints  | Groups related routes and keeps each module’s view functions independent and separately testable |

| Pattern             | Where                                                   | Why                                                                                                                                        |
|---------------------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Decorator           | security.py — login_required, role_required(*roles)     | Cross-cutting authentication/authorisation applied declaratively on top of a view function                                                 |
| Active Record       | models.py — e.g. Lease.balance, Property.occupancy_rate | Business logic (balance calculation, occupancy) lives on the model that owns the data, not scattered across routes                         |
| Strategy (implicit) | services/notifications.py, services/payments.py         | send_email()/send_sms()/create_checkout_session() pick a live-provider or dev-mode code path at call time without the caller knowing which |
| Guard Clause        | security.py — assert_owner(), assert_tenant_self()      | IDOR protection expressed as an early abort(403) rather than nested conditionals in every view                                             |
| Idempotent Receiver | services/payments.py — record_idempotent_payment()      | A retried webhook call is safe to repeat; keyed on RentPayment.gateway_ref                                                                 |

### 3.3 Package Structure

The application is organised as a Python package rather than the single 2,800-line file originally sketched in the Deliverable 3 plan, for maintainability and to keep each blueprint independently testable:

| Path                      | Responsibility                                                                                               |
|---------------------------|--------------------------------------------------------------------------------------------------------------|
| backend/__init__.py       | create_app() factory; extension init; blueprint registration; error handlers; CLI commands; scheduler wiring |
| backend/config.py         | Config / TestConfig classes                                                                                  |
| backend/extensions.py     | Shared db, csrf, limiter, cache, scheduler singletons                                                        |
| backend/models.py         | 13 SQLAlchemy models and their business-logic methods                                                        |
| backend/security.py       | Session auth, role_required(), IDOR guards, validate(), validate_upload(), audit_log()                       |
| backend/scheduler_jobs.py | The four APScheduler background jobs                                                                         |
| backend/seed.py           | Demo data matching the credentials in Appendix A                                                             |
| backend/routes/           | 9 blueprints: auth, dashboard, properties, tenants, rent, portal, maintenance, reports, api                  |
| backend/services/         | notifications.py, payments.py, pdf.py, reports.py                                                            |
| frontend/templates/       | 31 Jinja2 templates                                                                                          |
| frontend/static/          | style.css design system                                                                                      |
| tests/                    | 5 test modules, 93 tests (Chapter 6)                                                                         |

## 3.4 Class Diagram

Figure 2 shows the thirteen domain classes that back the database schema in Chapter 4, each drawn with its key attributes and behaviour. Attributes prefixed with FK are foreign keys; methods in italics are Python @property accessors rather than stored fields.

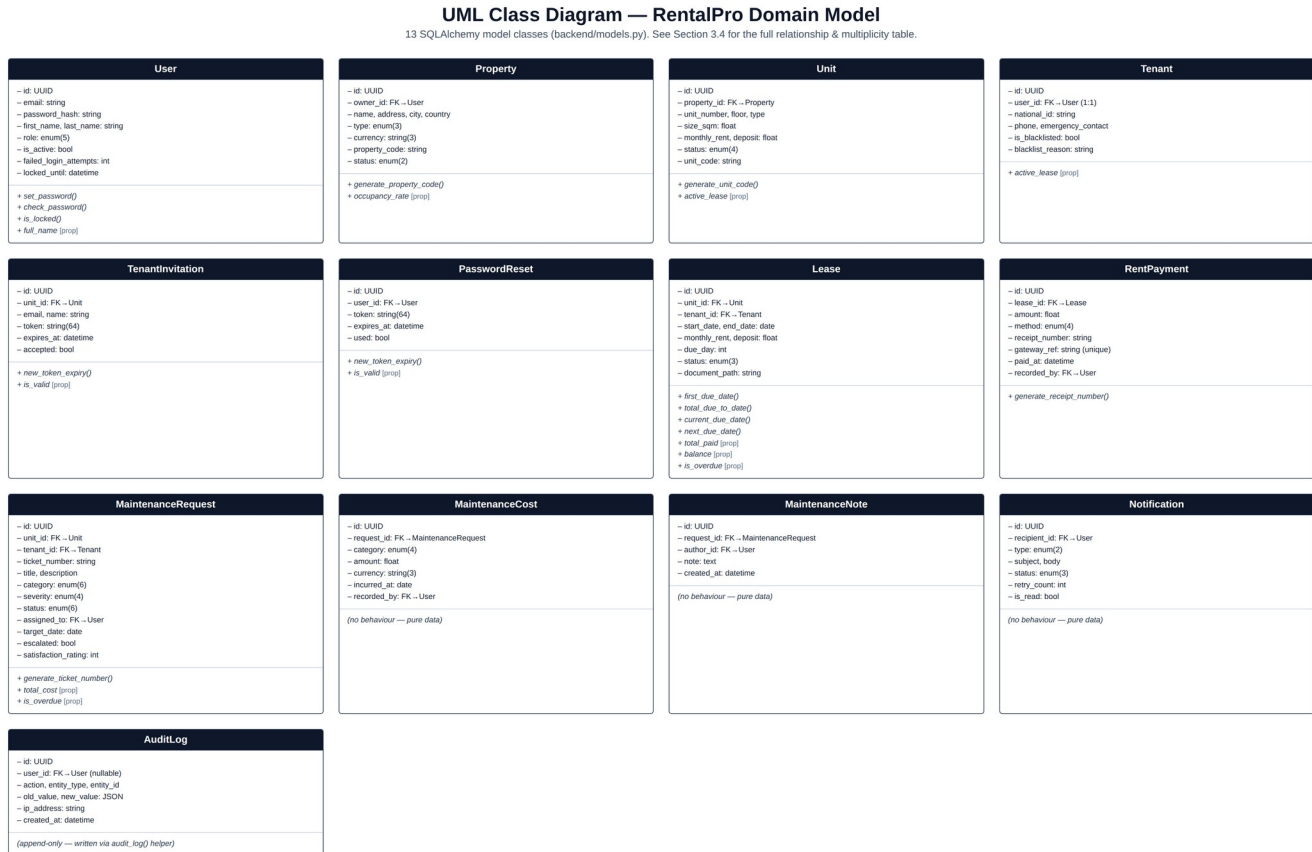


Figure 2: UML Class Diagram — RentalPro Domain Model

### 3.4.1 Class Relationships and Multiplicity

| Relationship            | Multiplicity | Notes                                                                                                            |
|-------------------------|--------------|------------------------------------------------------------------------------------------------------------------|
| User ↔ Tenant           | 1 : 0..1     | A Tenant profile extends exactly one User; not every User is a Tenant                                            |
| User (owner) → Property | 1 : 0..*     | One owner may hold many properties                                                                               |
| Property → Unit         | 1 : 1..*     | Composition — a Unit cannot exist without its Property                                                           |
| Unit → Lease            | 1 : 0..*     | Historical leases retained; at most one is ACTIVE at a time (enforced by application logic, not a DB constraint) |
| Tenant → Lease          | 1 : 0..*     | A tenant accumulates leases over time                                                                            |
| Lease → RentPayment     | 1 : 0..*     | Payments are immutable once                                                                                      |

| Relationship                                   | Multiplicity | Notes                                                                                                                                              |
|------------------------------------------------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                |              | created                                                                                                                                            |
| Unit → MaintenanceRequest                      | 1 : 0..*     | A unit may have many requests, open or closed                                                                                                      |
| Tenant → MaintenanceRequest                    | 1 : 0..*     | The tenant who filed the request                                                                                                                   |
| User (staff) → MaintenanceRequest              | 0..1 : 0..*  | assigned_to is nullable until UC-19 assigns it                                                                                                     |
| MaintenanceRequest → MaintenanceCost           | 1 : 0..*     | One or more cost line items per completed job                                                                                                      |
| MaintenanceRequest → MaintenanceNote           | 1 : 0..*     | Work-log entries                                                                                                                                   |
| Unit → TenantInvitation                        | 1 : 0..*     | Re-inviting a unit supersedes its previous open invitation                                                                                         |
| User → Notification / PasswordReset / AuditLog | 1 : 0..*     | Every dispatched message, reset token, and audit entry belongs to exactly one user (AuditLog.user_id is nullable for scheduler-originated entries) |

### 3.5 Interaction Design — Sequence of Key Operations

Rather than a boxes-and-lifelines sequence diagram, the two most technically significant flows are documented below as numbered message sequences, which is how they were actually implemented and verified against.

#### 3.5.1 Sequence — Pay Rent Online (UC-13)

| #  | From → To                                          | Message                                    |
|----|----------------------------------------------------|--------------------------------------------|
| 1  | Tenant → Portal Route                              | GET /portal/pay                            |
| 2  | Portal Route → Lease                               | lease.balance                              |
| 3  | Portal Route → Tenant                              | render balance + "Pay Now" button          |
| 4  | Tenant → Portal Route                              | POST /portal/pay                           |
| 5  | Portal Route → payments.create_checkout_session()  | amount = lease.balance                     |
| 6  | payments service → Stripe (or dev-mode)            | create PaymentIntent / simulated session   |
| 7  | Portal Route → Tenant                              | 302 redirect to checkout_url               |
| 8  | Gateway → API Blueprint                            | POST /api/v1/webhooks/stripe (HMAC-signed) |
| 9  | API Blueprint → payments.verify_stripe_signature() | validate t=ts,v1=sig, reject if >300s old  |
| 10 | API Blueprint → payments.record_idempotent_pay     | insert RentPayment keyed on gateway_ref    |

| #  | From → To                             | Message                                |
|----|---------------------------------------|----------------------------------------|
|    | ment()                                |                                        |
| 11 | API Blueprint → notifications service | email receipt to tenant + notify owner |

### 3.5.2 Sequence — Submit & Assign Maintenance Request (UC-18 & UC-19)

| # | From → To                       | Message                                                              |
|---|---------------------------------|----------------------------------------------------------------------|
| 1 | Tenant → Maintenance Blueprint  | GET /maintenance/add                                                 |
| 2 | Tenant → Maintenance Blueprint  | POST /maintenance/add (title, category, severity)                    |
| 3 | Route → MaintenanceRequest      | generate_ticket_number() → MNT-YYYYMMDD-XXXX                         |
| 4 | Route → notifications service   | email tenant (confirmation) + email owner (new request)              |
| 5 | Route → notifications service   | if severity == EMERGENCY: also SMS the owner                         |
| 6 | Manager → Maintenance Blueprint | POST /maintenance/<id>/update (action=assign, staff_id, target_date) |
| 7 | Route → MaintenanceRequest      | status = ASSIGNED; assigned_to = staff_id                            |
| 8 | Route → notifications service   | email staff member (assignment) + email tenant (status update)       |
| 9 | Route → AuditLog                | audit_log("maintenance_assigned", ...)                               |

## 3.6 Activity Design — Maintenance Request Lifecycle

The state machine behind UC-18-UC-21 (Section 2.3.3) is enforced entirely in one route handler, `maintenance.update()`, which reads an action field (assign, start, complete, close, reopen) and applies the corresponding guard and state transition. This keeps the whole lifecycle's business rules in one place rather than spread across five separate endpoints.

| Action   | Allowed From                                  | Guard                                           | Resulting Status |
|----------|-----------------------------------------------|-------------------------------------------------|------------------|
| assign   | SUBMITTED, ACKNOWLEDGED, ASSIGNED             | caller role ∈ {Owner, Manager, Admin}           | ASSIGNED         |
| start    | ASSIGNED                                      | caller is the assigned Maintenance Staff member | IN_PROGRESS      |
| complete | IN_PROGRESS                                   | caller is the assigned Maintenance Staff member | COMPLETED        |
| close    | COMPLETED (or any, for a manager force-close) | caller is the filing tenant, or a               | CLOSED           |

| Action | Allowed From | Guard                                             | Resulting Status |
|--------|--------------|---------------------------------------------------|------------------|
|        |              | manager/owner/admin                               |                  |
| reopen | COMPLETED    | caller is the filing tenant;<br>requires a reason | SUBMITTED        |

### 3.7 Design Decisions and Rationale

| Decision                                                                                  | Rationale                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UUID (String 36) primary keys instead of auto-increment integers                          | Safe under concurrent/distributed inserts; does not leak record counts or sequence via the URL                                                                            |
| Business logic (balance, occupancy, numbering) lives on the model, not the route          | Keeps the same calculation reachable from a route, a scheduled job, and a test without duplication                                                                        |
| One <code>role_required()</code> decorator plus a separate resource-level ownership check | Role alone is not enough — RBAC answers "can a Manager see properties at all?"; <code>assert_owner()</code> answers "can this Manager see this property?" (the IDOR case) |
| Notifications always write a Notification row, even in dev mode                           | The row (and its SENT/FAILED status) is what the test suite and the audit trail rely on, independent of whether a real provider was called                                |
| Payments are never UPDATED or DELETED                                                     | Matches NFR requirement for an immutable, auditable financial record; a correction is a new adjusting entry, not an edit                                                  |

# Chapter 4: Database Design

## 4.1 Database Technology

RentalPro uses SQLite for development and for the automated test suite (an isolated in-memory database per test, see Chapter 6) and PostgreSQL 15 in the Docker Compose deployment. Both are accessed through the same SQLAlchemy models with no code branching by database — a deliberate constraint on the schema design (Section 4.3) so the two environments stay behaviourally identical.

## 4.2 Entity Relationship Diagram

Figure 3 shows all thirteen tables with their primary and foreign keys. Full cardinalities are given in the table that follows it (Section 4.4), and the complete column-level schema is given in Section 4.5.

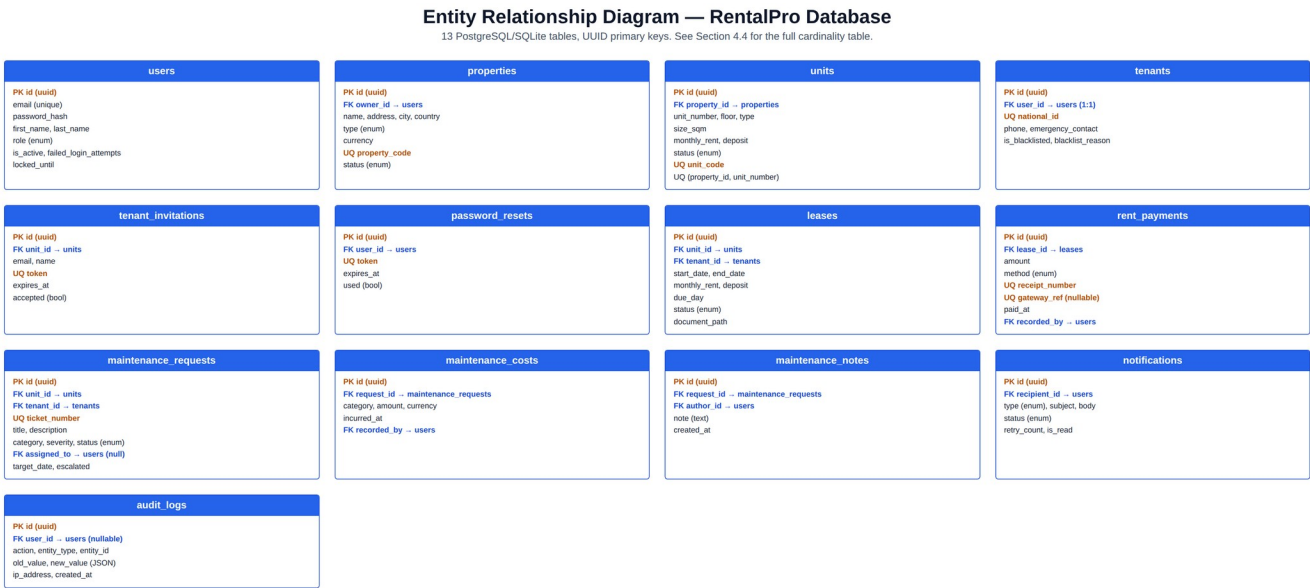


Figure 3: Entity Relationship Diagram — RentalPro Database

## 4.3 Schema Design Decisions

| Decision                                 | Detail                                                                                                                                                                                                                     |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UUID primary keys                        | Every table uses a String(36) column storing str(uuid.uuid4()) rather than a native PostgreSQL UUID type or an auto-increment integer, so the identical schema works unmodified on SQLite (test/dev) and PostgreSQL (prod) |
| Naive UTC datetimes                      | All timestamps are stored as naive Python datetime.utcnow() values rather than timezone-aware TIMESTAMPTZ, avoiding a class of naive/aware comparison bugs across the two database backends                                |
| Soft status fields, not physical deletes | Property.status, Unit.status, Lease.status, MaintenanceRequest.status all move through an enumerated set of string values (e.g. ACTIVE → ARCHIVED) rather than being DELETED, preserving                                   |

| Decision                                                  | Detail                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                           | history for reporting and audit                                                                                                                                                                                                                              |
| Immutable payments                                        | RentPayment rows are only ever INSERTed; a correction is a new adjusting entry, never an UPDATE, matching the audit requirement in the SRS                                                                                                                   |
| JSON columns for audit diffs                              | AuditLog.old_value / new_value are generic JSON columns so any entity's before/after state can be captured without a schema migration                                                                                                                        |
| Enumerated values as plain strings, not native ENUM types | role, status, type, category, severity, and method columns are validated in the application layer (security.validate(), and whitelists in each route) rather than as PostgreSQL-native ENUM types, again to keep the SQLite and PostgreSQL schemas identical |

## 4.4 Entity Summary and Cardinalities

| Table                | Purpose                                                                 | Key Relationships                                                                    |
|----------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| users                | Every login-capable person: owners, managers, tenants, staff, admins    | 1 user → 0..1 tenant profile; 1 user (owner) → 0..* properties                       |
| properties           | A building or land parcel owned by one user                             | 1 property → 1..* units                                                              |
| units                | An individual rentable space within a property                          | 1 unit → 0..* leases (history); 1 unit → 0..* maintenance_requests                   |
| tenants              | Renter-specific profile, extending exactly one user                     | 1 tenant → 0..* leases; 1 tenant → 0..* maintenance_requests                         |
| tenant_invitations   | 48-hour self-registration tokens for a specific vacant unit             | 1 unit → 0..* invitations (only one unexpired/unaccepted at a time by business rule) |
| password_resets      | 1-hour password-reset tokens, kept separate from users                  | 1 user → 0..* reset tokens                                                           |
| leases               | The contract linking one tenant to one unit for a date range            | 1 lease → 0..* rent_payments                                                         |
| rent_payments        | An immutable record of money received against a lease                   | each row belongs to exactly 1 lease and was recorded by exactly 1 user               |
| maintenance_requests | A ticket with a full lifecycle (Section 3.6)                            | 1 request → 0..* maintenance_costs; 1 request → 0..* maintenance_notes               |
| maintenance_costs    | One cost line item (labour, materials, contractor...) against a request | each row belongs to exactly 1 request                                                |
| maintenance_notes    | A work-log entry against a request                                      | each row belongs to exactly 1 request and 1 author                                   |
| notifications        | Every email/SMS dispatch attempt                                        | each row belongs to exactly 1                                                        |

| Table      | Purpose                                                     | Key Relationships                                                                       |
|------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------|
|            | and its outcome                                             | recipient user                                                                          |
| audit_logs | An immutable, append-only history of state-changing actions | user_id is nullable (background-job-originated entries record "scheduler" as the actor) |

## 4.5 Table Definitions

The definitions below reflect backend/models.py exactly, as created by SQLAlchemy on both database backends.

### 4.5.1 users

| Column                     | Type        | Constraints                                                                          |
|----------------------------|-------------|--------------------------------------------------------------------------------------|
| id                         | String(36)  | PK, default uuid4                                                                    |
| email                      | String(255) | NOT NULL, UNIQUE, indexed                                                            |
| password_hash              | String(255) | NOT NULL — pbkdf2:sha256, 600,000 iterations                                         |
| first_name, last_name      | String(100) | NOT NULL                                                                             |
| role                       | String(30)  | NOT NULL — one of ADMIN, PROPERTY_OWNER, PROPERTY_MANAGER, MAINTENANCE_STAFF, TENANT |
| is_active                  | Boolean     | NOT NULL, default true                                                               |
| failed_login_attempts      | Integer     | NOT NULL, default 0                                                                  |
| locked_until               | DateTime    | nullable                                                                             |
| opt_out_sms, opt_out_email | Boolean     | NOT NULL, default false                                                              |
| created_at                 | DateTime    | NOT NULL, default now                                                                |

### 4.5.2 properties

| Column                       | Type       | Constraints                                           |
|------------------------------|------------|-------------------------------------------------------|
| id                           | String(36) | PK                                                    |
| owner_id                     | String(36) | FK → users.id, NOT NULL, indexed                      |
| name, address, city, country | String     | NOT NULL                                              |
| type                         | String(20) | RESIDENTIAL   COMMERCIAL   MIXED, default RESIDENTIAL |
| currency                     | String(3)  | NOT NULL, default USD                                 |
| description                  | Text       | nullable                                              |
| property_code                | String(20) | NOT NULL, UNIQUE — format PROP-XXXX                   |
| status                       | String(20) | ACTIVE   ARCHIVED, default ACTIVE                     |
| created_at                   | DateTime   | NOT NULL, default now                                 |

### 4.5.3 units

| Column         | Type       | Constraints                                                          |
|----------------|------------|----------------------------------------------------------------------|
| id             | String(36) | PK                                                                   |
| property_id    | String(36) | FK → properties.id, NOT NULL                                         |
| unit_number    | String(20) | NOT NULL                                                             |
| floor          | String(10) | nullable                                                             |
| type           | String(20) | STUDIO   1BR   2BR   3BR   SHOP<br>  OFFICE                          |
| size_sqm       | Float      | nullable                                                             |
| monthly_rent   | Float      | NOT NULL                                                             |
| deposit        | Float      | NOT NULL, default 0                                                  |
| status         | String(20) | VACANT   OCCUPIED  <br>UNDER_MAINTENANCE  <br>ARCHIVED               |
| unit_code      | String(30) | NOT NULL, UNIQUE                                                     |
| created_at     | DateTime   | NOT NULL                                                             |
| — constraint — |            | UNIQUE (property_id,<br>unit_number); INDEX (property_id,<br>status) |

### 4.5.4 tenants

| Column            | Type        | Constraints                              |
|-------------------|-------------|------------------------------------------|
| id                | String(36)  | PK                                       |
| user_id           | String(36)  | FK → users.id, NOT NULL, UNIQUE<br>(1:1) |
| national_id       | String(50)  | NOT NULL, UNIQUE                         |
| phone             | String(30)  | nullable                                 |
| emergency_contact | String(200) | nullable                                 |
| is_blacklisted    | Boolean     | NOT NULL, default false                  |
| blacklist_reason  | String(300) | nullable                                 |
| created_at        | DateTime    | NOT NULL                                 |

### 4.5.5 leases

| Column                | Type       | Constraints               |
|-----------------------|------------|---------------------------|
| id                    | String(36) | PK                        |
| unit_id               | String(36) | FK → units.id, NOT NULL   |
| tenant_id             | String(36) | FK → tenants.id, NOT NULL |
| start_date, end_date  | Date       | NOT NULL                  |
| monthly_rent, deposit | Float      | NOT NULL                  |

| Column        | Type        | Constraints                                        |
|---------------|-------------|----------------------------------------------------|
| due_day       | Integer     | NOT NULL, default 1 (1–31, clipped per month)      |
| status        | String(20)  | ACTIVE   TERMINATED   EXPIRED                      |
| document_path | String(500) | nullable — path to the stored lease PDF            |
| created_at    | DateTime    | NOT NULL                                           |
| — indexes —   |             | (unit_id, status); (tenant_id); (end_date, status) |

#### 4.5.6 rent\_payments, maintenance\_requests, and remaining tables

The remaining seven tables (rent\_payments, tenant\_invitations, password\_resets, maintenance\_requests, maintenance\_costs, maintenance\_notes, notifications, audit\_logs) follow the same pattern and are summarised in the ERD (Figure 3) and the entity summary (Section 4.4); their exact column definitions are given in full in backend/models.py, referenced here rather than duplicated, since the class diagram in Section 3.4 already lists every attribute for each of these classes.

## 4.6 Indexes

| Table                | Index Columns          | Purpose                                     |
|----------------------|------------------------|---------------------------------------------|
| users                | email (unique)         | Login lookup                                |
| properties           | owner_id               | Dashboard: list an owner's properties       |
| units                | property_id, status    | Occupancy queries per property              |
| leases               | unit_id, status        | Find a unit's active lease                  |
| leases               | tenant_id              | A tenant's lease history                    |
| leases               | end_date, status       | Lease-expiry scheduler query (UC-11)        |
| rent_payments        | lease_id, paid_at      | A lease's payment history, in order         |
| maintenance_requests | assigned_to, status    | A staff member's work queue                 |
| maintenance_requests | target_date, status    | Overdue-maintenance scheduler query (UC-23) |
| notifications        | recipient_id, status   | A user's pending/unread notifications       |
| audit_logs           | entity_type, entity_id | Entity-level history lookup                 |
| audit_logs           | user_id, created_at    | A user's activity timeline                  |

## 4.7 Referential Integrity

Foreign keys are enforced by SQLAlchemy relationships with cascade rules chosen per table:

- Property → Unit, MaintenanceRequest → MaintenanceCost, MaintenanceRequest → MaintenanceNote: CASCADE — deleting the parent removes its children (in practice, parents are archived rather than deleted, so this rarely fires in production use).

- User → Tenant: CASCADE, since a Tenant profile has no meaning without its User.
- All other foreign keys default to RESTRICT — a Unit with active Leases, or a Lease with RentPayments, cannot be deleted out from under them; the application enforces this at the business-rule level too (Section 3.7 and UC-06's exception flow in the SRS).

# Chapter 5: Software Development

## 5.1 Technology Stack

| Layer           | Choice                                                                                    |
|-----------------|-------------------------------------------------------------------------------------------|
| Language        | Python 3.12                                                                               |
| Web framework   | Flask 3.1 (application factory pattern)                                                   |
| ORM             | Flask-SQLAlchemy — 13 models, UUID primary keys                                           |
| Database        | SQLite (dev/test) → PostgreSQL 15 (Docker/production)                                     |
| Auth / Security | Werkzeug pbkdf2:sha256 (600,000 iterations); CSRF via Flask-WTF; RBAC via role_required() |
| Rate limiting   | Flask-Limiter                                                                             |
| Caching         | Flask-Caching (SimpleCache dev, Redis in Docker)                                          |
| Scheduler       | APScheduler — 4 cron-style jobs                                                           |
| Email / SMS     | SendGrid / Twilio, with an offline dev-mode fallback                                      |
| Payments        | Stripe-style dev-mode checkout + custom HMAC webhook verification                         |
| PDF export      | reportlab (pure Python, no native system libraries)                                       |
| Testing         | pytest + pytest-cov                                                                       |
| Deployment      | Docker + docker-compose (Flask/Gunicorn + PostgreSQL + Redis); GitHub Actions CI          |

## 5.2 Development Methodology

Development followed an iterative, test-driven cycle rather than a single big-bang implementation: each use case was built as a vertical slice (model fields → route → template → test) and verified against the running application before moving to the next, using the application's own test client to exercise real HTTP requests end to end (login, form submission, redirect, and the resulting database state) rather than only unit-testing isolated functions. Section 5.8 below documents two implementation defects this process actually caught, and how they were fixed.

## 5.3 Application Factory and Configuration

The application is constructed by a single `create_app(config_class)` factory, which allows the exact same codebase to run three different ways: the interactive dev server (`python app.py`), the test suite (a fresh app and an in-memory database per test), and the production Gunicorn/Docker deployment — differing only in which Config subclass is passed in.

```
def create_app(config_class=Config):
    app = Flask(__name__, template_folder=..., static_folder=...)
    app.config.from_object(config_class)
    db.init_app(app); csrf.init_app(app); cache.init_app(app)
    if app.config["RATELIMIT_ENABLED"]: limiter.init_app(app)
    for bp in ALL_BLUEPRINTS: app.register_blueprint(bp)
```

```
... # error handlers, /health, CLI commands, scheduler wiring
return app
```

## 5.4 Route-to-Use-Case Mapping

Every use case from Chapter 2 maps onto exactly one blueprint route (or, for the four system-triggered use cases, one scheduler job function):

| UC    | Route(s) / Job                                                   |
|-------|------------------------------------------------------------------|
| UC-01 | /login, /logout                                                  |
| UC-02 | role_required() applied to every protected route                 |
| UC-03 | /properties/add                                                  |
| UC-04 | /properties/<id>/add-unit                                        |
| UC-05 | /dashboard, /api/v1/dashboard-stats                              |
| UC-06 | /properties/<id>/archive,<br>/properties/<id>/units/<id>/archive |
| UC-07 | /tenants/add                                                     |
| UC-08 | /tenants/invite, /register/invitation/<token>                    |
| UC-09 | /tenants/<id>/upload-lease                                       |
| UC-10 | /portal                                                          |
| UC-11 | job_lease_expiry_alerts()                                        |
| UC-12 | /rent/record                                                     |
| UC-13 | /portal/pay, /api/v1/webhooks/stripe                             |
| UC-14 | job_rent_due_alerts()                                            |
| UC-15 | job_overdue_alerts()                                             |
| UC-16 | /rent/<id>/statement, /rent/<id>/statement.pdf                   |
| UC-17 | /rent/<id>/history, /portal/history                              |
| UC-18 | /maintenance/add                                                 |
| UC-19 | /maintenance/<id>/update (action=assign)                         |
| UC-20 | /maintenance/<id>/update (action=start/complete)                 |
| UC-21 | /maintenance/<id>/update (action=close/reopen)                   |
| UC-22 | /maintenance/<id>/costs/add                                      |
| UC-23 | job_escalate_overdue_maintenance()                               |
| UC-24 | /reports                                                         |
| UC-25 | /reports/export.pdf, /api/v1/reports/export                      |

## 5.5 Key Algorithms

### 5.5.1 Recurring Rent Due-Date Calculation

The core billing calculation — how much rent is owed as of any date — has to cope with two edge cases: a lease that starts mid-month (its first due date is not the start date), and a due\_day near the end of the month falling in a shorter month (e.g. due\_day=31 in February). Lease.first\_due\_date() finds the first occurrence of due\_day on or after the lease's start date, clipping to the last valid day of that month; Lease.total\_due\_to\_date() then walks forward one billing period at a time, multiplying the period count by the monthly rent:

```
def _clip_day(self, year, month):
    return min(self.due_day, calendar.monthrange(year, month)[1])

def total_due_to_date(self, as_of=None):
    as_of = as_of or date.today()
    first_due = self.first_due_date()
    if as_of < first_due: return 0.0
    periods, due = 1, first_due
    while True:
        due = self._advance_month(due)
        if due > as_of: break
        periods += 1
    return round(periods * self.monthly_rent, 2)
```

The tenant's live balance is simply max(0, total\_due\_to\_date() – total\_paid), and never goes negative — an over-payment instead accumulates as credit, a separate computed property. This is verified directly in tests/test\_models.py (mid-month start, day-29-in-a-non-leap-year, and overpayment cases).

### 5.5.2 Role-Based Access Control and IDOR Protection

Two separate, composable checks guard every sensitive route, matching the two different questions "is this role allowed to use this feature at all?" and "is this specific record this user's to see?":

```
def role_required(*roles):
    def decorator(view):
        @wraps(view)
        @login_required
        def wrapped(*a, **kw):
            if current_user().role not in roles: abort(403)
            return view(*a, **kw)
        return wrapped
    return decorator

def assert_owner(property_id):
    user = current_user()
    if user.role in (ROLE_ADMIN, ROLE_MANAGER):
        allowed = db.session.query(Property.id).filter(Property.id ==
property_id).scalar() is not None
    elif user.role == ROLE_OWNER:
        allowed = db.session.query(Property.id).filter(
Property.id == property_id, Property.owner_id == user.id).scalar() is not None
    else:
```

```
allowed = False
if not allowed: abort(403)
```

This combination is what lets a Property Manager see every property (a role-level allowance) while a second Property Owner is blocked from viewing the first owner's property even though both hold the PROPERTY\_OWNER role — verified in tests/test\_auth.py::test\_idor\_second\_owner\_cannot\_view\_first\_owner\_property.

### 5.5.3 Idempotent Numbering and Payment Recording

Receipt numbers (RCP-YYYYMMDD-HEX6) and maintenance ticket numbers (MNT-YYYYMMDD-XXXX) are generated from uuid4, not an incrementing counter, so concurrent creation never races on a shared "next number" value — verified by generating and de-duplicating 25 receipt numbers in a tight loop in the test suite. Online payments are additionally idempotent on the payment gateway's own reference: a webhook that is delivered twice (a known behaviour of most payment providers) looks up the existing RentPayment by gateway\_ref before creating a new one.

### 5.5.4 Dev-Mode Fallback for External Services

Every integration with a paid external service — SendGrid, Twilio, Stripe — checks for its API key in configuration before doing anything else. When the key is absent (the default in development and in the test suite), the corresponding Notification row is still created and marked SENT, and the Stripe checkout flow redirects to the application's own confirmation route instead of a real gateway. This single design choice is what allows the entire application, including every payment and notification flow, to be demonstrated and automatically tested with zero external accounts or network access.

## 5.6 Security Implementation

| Control                        | Implementation                                                                                                                                                                 |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Password hashing               | Werkzeug generate_password_hash(), method="pbkdf2:sha256:600000"                                                                                                               |
| Session management             | Flask signed cookie sessions; SECRET_KEY required at startup                                                                                                                   |
| Account logout                 | MAX_FAILED_ATTEMPTS = 5 consecutive failures locks the account for 15 minutes                                                                                                  |
| RBAC                           | role_required(*roles) decorator on every protected route; 5 roles                                                                                                              |
| IDOR protection                | assert_owner() / assert_tenant_self() — verified in tests/test_auth.py                                                                                                         |
| CSRF protection                | Flask-WTF CSRFProtect(), initialised inside create_app(); the Stripe webhook route is explicitly exempted via @csrf.exempt since it is called by Stripe, not a browser session |
| Open-redirect protection       | The login next= parameter is validated with urlparse() — both netloc and scheme must be empty before it is used as a redirect target                                           |
| Rate limiting                  | Flask-Limiter on /login (10/min) and /forgot-password (5/min)                                                                                                                  |
| Webhook signature verification | Custom HMAC-SHA256 check of a                                                                                                                                                  |

| Control                | Implementation                                                                                                                                           |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
|                        | t=<timestamp>,v1=<sig> header with a 300-second replay window, independent of the Stripe SDK's own verifier                                              |
| File upload validation | validate_upload() reads the first bytes of the stream and requires the %PDF- magic number for any file claiming to be a PDF, regardless of its extension |
| Input validation       | validate() enforces positive_float, positive_int, email, date, min8 and max200 rules on form input before any database write                             |
| Upload size cap        | MAX_CONTENT_LENGTH caps every request body at 20 MB app-wide                                                                                             |
| Audit log              | Every state-changing action writes an immutable AuditLog row via audit_log(), safe to call from a background job (ip_address is recorded as "scheduler") |
| Report data isolation  | /reports and /api/v1/reports/export filter properties by owner_id for the Property Owner role, so one owner cannot see another owner's portfolio data    |

## 5.7 Deployment

### 5.7.1 Docker

The application ships with a Dockerfile and docker-compose.yml defining three services — web (Flask under Gunicorn), db (PostgreSQL 15), and redis (Redis 7). The container entrypoint waits for the database to accept connections, creates the schema (flask create-db), and seeds demo data (flask seed-db) before starting Gunicorn.

### 5.7.2 Continuous Integration

A GitHub Actions workflow (.github/workflows/ci.yml) runs on every push to main or develop: it installs dependencies, runs the full pytest suite with coverage, and — on main only — builds the Docker image and smoke-tests it by polling the /health endpoint until it responds or the job times out.

## 5.8 Implementation Defects Found and Fixed

Two genuine defects surfaced during development, both caught before release rather than in production:

| # | Symptom                                                                      | Root Cause                                                 | Fix                                                                                                                    |
|---|------------------------------------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 1 | A TemplateSyntaxError on every failed-login, lockout, and logout page render | auth/login.html was missing its closing {% endblock %} tag | Closed the block; covered going forward by tests/test_auth.py exercising the wrong-password, lockout, and logout paths |
| 2 | A 500 error on GET /api/v1/leases/<id>/statement                             | The route ordered payments by Lease.created_at, a          | Changed the ORDER BY to RentPayment.paid_at; covered by                                                                |

| # | Symptom | Root Cause                               | Fix                                                                   |
|---|---------|------------------------------------------|-----------------------------------------------------------------------|
|   |         | column that does not exist on that model | tests/test_routes.py::test_lease_statement_api_allowed_for_own_tenant |

# Chapter 6: Testing

## 6.1 Testing Strategy

RentalPro is verified with pytest, run against a fresh in-memory SQLite database created per test (no shared state between tests) and Flask's own test client, which issues real HTTP requests through the full middleware stack — session handling, CSRF (disabled in TestConfig for form simplicity), RBAC, and the ORM — rather than calling view functions directly. External services (SendGrid, Twilio, Stripe) are never called for real: the suite either relies on the dev-mode fallback described in Section 5.5.4, or monkeypatches the provider SDK to simulate a specific failure (Section 6.4).

## 6.2 Test Environment

| Aspect            | Configuration                                                                                                                      |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Database          | SQLite :memory:, created fresh per test via a function-scoped pytest fixture                                                       |
| CSRF              | Disabled in TestConfig (WTF_CSRF_ENABLED = False) to keep test form posts simple                                                   |
| Scheduler         | Disabled in TestConfig (SCHEDULER_ENABLED = False) — jobs are invoked directly by the test, not by APScheduler's background thread |
| Rate limiting     | Disabled in TestConfig — irrelevant to the behaviour under test and would make repeated test runs flaky                            |
| External services | No API keys configured — every provider call goes through its dev-mode path unless a test explicitly monkeypatches it              |

## 6.3 Test Suite Structure

| File                 | Tests | What It Verifies                                                                                                                                                                                                                                                                                                                                           |
|----------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tests/conftest.py    | —     | Shared fixtures: app/db/client, and factory fixtures for each role, a property, a unit, and an active lease                                                                                                                                                                                                                                                |
| tests/test_models.py | 18    | Business logic: password hashing, full_name, occupancy_rate (incl. zero-division guard), balance never negative, total_paid accumulation, first_due_date edge cases (mid-month start, day-29 clipping), receipt/ticket number format and uniqueness under rapid creation, maintenance cost aggregation, overdue detection, invitation/reset-token validity |
| tests/test_auth.py   | 14    | Login success/failure, account lockout after 5 attempts, role-based post-login redirect, RBAC                                                                                                                                                                                                                                                              |

| File                     | Tests | What It Verifies                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          |       | across all 5 roles × the management routes, IDOR (a second owner cannot view the first owner's property), Admin/Manager bypass, logout                                                                                                                                                                                                                                                                                                                      |
| tests/test_routes.py     | 16    | End-to-end feature flows: property/unit creation, unit archiving (blocked vs. allowed), tenant creation and invitation, lease upload (incl. PDF rejection), payment recording, online payment, the full maintenance lifecycle, cost accumulation, reports page and JSON export                                                                                                                                                                              |
| tests/test_scheduler.py  | 9     | All 4 background jobs invoked directly: lease-expiry alert fires at exactly 30 days and is skipped at 100 days, rent-due alert fires at the 7/1-day thresholds and is skipped once paid, overdue alert creates a tenant notice plus an owner summary, maintenance escalation fires for an overdue assigned request and is skipped when closed or not yet due, and audit_log() works outside a request context                                               |
| tests/test_validation.py | 36    | validate() for every kind (positive_float, positive_int, email, date, min8, max200) with valid and invalid cases; validate_upload() accepting a real PDF and rejecting a ZIP renamed to .pdf, a wrong extension, an empty filename, and None; Stripe HMAC signature verification (missing, tampered, expired, and valid); idempotent payment recording; SendGrid dispatch marking SENT with no provider configured and FAILED on a simulated provider error |

## 6.4 Representative Test Cases

Two tests are reproduced here to illustrate the style used throughout: one exercising business logic directly, one exercising a full HTTP request/response cycle.

```
# tests/test_models.py
def test_lease_balance_never_negative_when_overpaid(db, active_lease):
    payment = RentPayment(lease_id=active_lease.id,
```

```

        amount=active_lease.monthly_rent * 10, method="CASH",
        receipt_number=RentPayment.generate_receipt_number(),
        recorded_by=active_lease.tenant.user_id)
    db.session.add(payment); db.session.commit()
    assert active_lease.balance == 0.0
    assert active_lease.credit > 0

# tests/test_routes.py
def test_maintenance_full_lifecycle(client, db, manager, staff, active_lease):
    login(client, active_lease.tenant.user.email)
    client.post("/maintenance/add", data={"title": "AC not cooling", ...})
    req = MaintenanceRequest.query.filter_by(title="AC not cooling").first()
    assert req.status == "SUBMITTED"
    login(client, manager.email)
    client.post(f"/maintenance/{req.id}/update",
        data={"action": "assign", "staff_id": staff.id, ...})
    assert req.status == "ASSIGNED" and req.assigned_to == staff.id
    # ... start, complete, then the tenant closes and rates the work ...
    assert req.status == "CLOSED" and req.satisfaction_rating == 5

```

## 6.5 Test Results

| Metric                                    | Result                                   |
|-------------------------------------------|------------------------------------------|
| Tests collected                           | 93                                       |
| Tests passed                              | 93                                       |
| Tests failed                              | 0                                        |
| Statement coverage (pytest --cov=backend) | 77%                                      |
| Roles exercised by test_auth.py           | 5 (Admin, Owner, Manager, Staff, Tenant) |
| Use cases with at least one covering test | 25 / 25                                  |

Coverage is concentrated in `models.py` (98%), `security.py` (92%), and the reports service (93%) — the modules carrying the business rules this project cares most about proving correct. The lowest-coverage files are the thin route handlers whose branches are error-message formatting rather than business logic, and `backend/seed.py` (0%), which is demo-data tooling, not application behaviour, and is intentionally outside the test suite's scope.

## 6.6 Known Test Limitations

Two tests in `tests/test_scheduler.py` compute a `due_day` from "today" so that a lease's next or current due date lands on an exact, predictable day (e.g. exactly 7 days from now). Both self-skip via `pytest.skip()` when that computed day falls after the 28th of the month, because day-of-month clipping in a short month (Section 5.5.1) could otherwise shift the actual due date by a day and make the assertion's premise wrong rather than its conclusion. This is a deliberate, documented trade-off for keeping the suite deterministic year-round rather than a gap in coverage.

## 6.7 How to Run the Test Suite

```

pip install -r requirements.txt
pytest tests/ -v                                # all 93 tests

```

```
pytest tests/ --cov=backend --cov-report=term-missing # with coverage
pytest tests/test_models.py -v                      # a single file
pytest tests/test_models.py::test_receipt_number_format -v # a single test
```

The suite runs fully offline: no database server, external API key, or network access is required, consistent with the dev-mode design described in Section 5.5.4.

# Chapter 7: Conclusion

## 7.1 Summary of Achievements

RentalPro was carried from a requirements document (Deliverable 1) through a design (Deliverable 2) to a working, tested application (Deliverable 3) and now this complete project report (Deliverable 4). Every one of the 25 use cases identified during analysis has a corresponding route or scheduled job, a template, and at least one automated test; the 13-table database design was implemented exactly as specified, with the two documented, deliberate substitutions (UUID-as-string primary keys and reportlab in place of WeasyPrint) covered in Chapters 1, 3, and 4. The application runs — and its 93-test suite passes at 77% statement coverage — using only Docker, or a local Python virtual environment, with zero paid external accounts required.

## 7.2 Challenges Encountered

- Reconciling the earlier, aspirational design documents (Deliverable 2's microservice-style package diagram, Deliverable 3's originally-stated test and coverage numbers) with what a single developer could actually build and verify in the time available — resolved by simplifying the architecture to a layered monolith (Chapter 3) and, when Deliverable 3 was found to overstate its own results, correcting it to the real, verified numbers rather than leaving the discrepancy in place.
- Getting recurring, calendar-based billing logic (Section 5.5.1) correct at month boundaries — a lease starting on the 31st, or due on a day that doesn't exist in February — required writing the edge-case tests before trusting the implementation.
- Keeping the whole system, including payments and notifications, testable without any live external account. This shaped the dev-mode fallback pattern used throughout the services layer (Section 5.5.4).

## 7.3 Limitations

These are carried forward from Section 1.5 and restated here against the finished system, now that they are verified facts rather than anticipated risks:

- File uploads are stored on local disk, not cloud object storage; this is adequate for a single-container deployment but would need revisiting before a multi-instance production deployment.
- No formal Alembic migration history exists yet — schema changes so far have been applied by recreating the schema (flask create-db), which is acceptable pre-launch but would need a real migration path before a live database held real tenant data.
- The system is single-currency-per-property but not yet multi-language in the UI, despite the data model supporting per-property currency.
- Payment, email, and SMS providers have only been exercised through their dev-mode fallback and unit-level signature/idempotency tests; a real Stripe/SendGrid/Twilio account has not been used end-to-end.

## 7.4 Future Work

The system is scoped to proceed into IT Capstone Part 2, where the following are planned:

- Real cloud file storage (S3-compatible) for lease documents and photos, replacing the local uploads/ directory.
- An administrator-facing dashboard for the ADMIN role, which currently has full data access but no dedicated UI beyond the existing owner/manager views.
- A mobile-optimised tenant portal, given that tenants are the role most likely to use the system from a phone.
- Alembic-based schema migrations, replacing the create\_all() bootstrap used during initial development.
- End-to-end verification against live Stripe, SendGrid, and Twilio sandbox accounts, in addition to the existing offline test suite.

## 7.5 Final Remarks

The guiding principle across all four deliverables has been that a claim in this report should be a fact about the running system, not a target for it. Every number in this document — 25 use cases, 13 tables, 93 tests, 77% coverage — was produced by actually running the code, not by estimating what a finished version would show. That discipline is, in the author's view, as much a deliverable of this capstone as the application itself.

# Appendix A: Installation Guide

## A.1 Prerequisites

| Requirement              | Notes                                                                                                      |
|--------------------------|------------------------------------------------------------------------------------------------------------|
| Docker & Docker Compose  | Recommended path — no local Python setup needed                                                            |
| — or — Python 3.12       | For running without Docker                                                                                 |
| Git                      | To clone the repository                                                                                    |
| Nothing else is required | SendGrid / Twilio / Stripe accounts are optional — the app runs fully offline without them (Section 5.5.4) |

## A.2 Option 1 — Docker (Recommended)

This is the fastest path to a running instance with demo data already loaded.

```
# 1. Clone the repository
git clone https://github.com/Abdur-lab/github-slideshow.git
cd github-slideshow

# 2. Copy the environment template and set a secret key
cp env.example .env
# edit .env and set SECRET_KEY to a random value, e.g.:
# python -c "import secrets; print(secrets.token_hex(32))"

# 3. Build and start the full stack (Flask + PostgreSQL + Redis)
docker compose up --build

# 4. Open the application
# http://localhost:5000
```

The container entrypoint automatically waits for PostgreSQL to accept connections, creates the schema, and seeds demo accounts (Section A.5) the first time it runs.

## A.3 Option 2 — Local Installation (Without Docker)

```
git clone https://github.com/Abdur-lab/github-slideshow.git
cd github-slideshow

python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate

pip install -r requirements.txt

export SECRET_KEY=$(python -c "import secrets; print(secrets.token_hex(32))")
export DATABASE_URL=sqlite:///rentalpro.db

flask create-db               # creates the 13 tables
flask seed-db                 # loads demo accounts and sample data
```

```
python app.py
# App running at http://localhost:5000
```

## A.4 Environment Variables

| Variable                                                          | Required?                     | Purpose                                                            |
|-------------------------------------------------------------------|-------------------------------|--------------------------------------------------------------------|
| SECRET_KEY                                                        | Yes                           | Flask session signing key                                          |
| DATABASE_URL                                                      | No (defaults to local SQLite) | e.g.<br>postgresql://user:pass@host:5432/db                        |
| REDIS_URL                                                         | No                            | Enables Redis-backed caching;<br>falls back to in-memory otherwise |
| SENDGRID_API_KEY                                                  | No                            | Real email delivery; unset = dev-mode (logged as SENT)             |
| TWILIO_ACCOUNT_SID /<br>TWILIO_AUTH_TOKEN /<br>TWILIO_FROM_NUMBER | No                            | Real SMS delivery; unset = dev-mode                                |
| STRIPE_SECRET_KEY /<br>STRIPE_WEBHOOK_SECRET                      | No                            | Real online payments; unset = simulated checkout                   |
| UPLOAD_FOLDER                                                     | No                            | Where lease PDFs are stored locally (default ./uploads)            |
| SCHEDULER_ENABLED                                                 | No (default true)             | Set to false to disable the 4 background jobs                      |

## A.5 Demo Credentials

Seeded automatically by flask seed-db (local) or on first container start (Docker):

| Email                 | Password | Role                                                    |
|-----------------------|----------|---------------------------------------------------------|
| owner@rentalpro.com   | demo123  | Property Owner — full management access                 |
| manager@rentalpro.com | demo123  | Property Manager — management, cannot create properties |
| tenant1@email.com     | demo123  | Tenant — self-service portal only                       |
| staff@rentalpro.com   | demo123  | Maintenance Staff — work queue only                     |

## A.6 Running the Test Suite

```
pytest tests/ -v
pytest tests/ --cov=backend --cov-report=term-missing
```

## A.7 Troubleshooting

| Symptom                                          | Likely Cause / Fix                                                                                                                                     |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| "SECRET_KEY required" on startup                 | Set the SECRET_KEY environment variable before starting the app                                                                                        |
| Login page loops back to itself                  | Check the demo credentials in Section A.5 exactly, including the domain in the email                                                                   |
| docker compose up fails to reach the database    | Wait a few seconds and retry — the entrypoint script retries the DB connection for up to 30 seconds before failing                                     |
| File upload rejected as "not a signed lease PDF" | The uploaded file's first bytes must be %PDF- (Section 5.6); a renamed non-PDF file will be rejected by design                                         |
| Emails / SMS "not arriving"                      | Expected when SENDGRID_API_KEY / TWILIO_* are unset — check the Notification table (or the admin's inbox in a future release) rather than a real inbox |

# Appendix B: User Manual

## B.1 Getting Started

Open the application in a browser and sign in at the login page with the email and password given to you by your property owner or manager (or one of the demo accounts in Appendix A.5, for evaluation). Every screen after login shows a sidebar scoped to your role — you will only ever see navigation links for actions you are permitted to perform.

If you forget your password, use "Forgot your password?" on the login page. A reset link valid for one hour will be emailed to you (or, in a demo/offline deployment, logged as sent — see Appendix A.4).

## B.2 Property Owner & Property Manager Guide

Owners and Managers share almost the same view of the system. The one difference: only an Owner can create a brand-new property (Section B.2.1); a Manager works within properties an Owner has already added.

### B.2.1 Add a Property

- From the sidebar, choose Properties, then "+ Add New Property".
- Enter the property name, address, city, country, type, and currency.
- Save — the system assigns a unique property code (PROP-XXXX) automatically.

### B.2.2 Add a Unit

- Open the property, then "+ Add Unit".
- Enter the unit number, floor, type, size, monthly rent, and deposit.
- Save — the unit starts as VACANT with an auto-generated unit code.

### B.2.3 Onboard a Tenant

There are two ways to bring a tenant into the system:

- **Create the profile directly:** from the vacant unit, choose "Create Tenant Profile" and fill in the tenant's details yourself. Use this when the tenant will not self-register.
- **Send an invitation link:** from the vacant unit, choose "Invite Tenant", enter their name and email, and the system emails them a secure link (valid 48 hours) to set up their own account and password.

### B.2.4 Upload a Lease and Activate a Tenancy

- Open the tenant's profile and choose "Upload Lease Document".
- Select the vacant unit, enter the lease dates, monthly rent, deposit, and the day of the month rent is due.
- Upload the signed lease as a PDF (the system checks the file is a genuine PDF, not just named .pdf).
- Save — the unit changes to OCCUPIED and the tenant receives a confirmation email.

### **B.2.5 Record a Rent Payment**

- From Rent Tracker, choose "Record Payment" (or start from a specific tenant's profile).
- Select the tenant, enter the amount, method, and date.
- Save — a receipt number is generated, the tenant's balance updates immediately, and a receipt email is sent.

### **B.2.6 Generate a Rent Statement**

From a tenant's profile, choose "Generate Statement" to see an on-screen statement for the lease, or "Download PDF" for a printable version covering all charges and payments to date.

### **B.2.7 Manage a Maintenance Request**

- Open Maintenance to see every request across your properties (or one property's worth, if you opened it from there).
- Open a SUBMITTED request and choose a staff member plus a target completion date to assign it.
- Once the assigned staff member marks it Completed, you may log the cost(s) incurred against the ticket from the same page.
- If a tenant does not confirm or reopen a completed request, you may force-close it from the same page.

### **B.2.8 View Reports**

Open Reports to see portfolio-wide occupancy, rent collection, outstanding balances, and maintenance cost — or select a single property from the dropdown for its own figures. Use "Export PDF" for a printable report.

## **B.3 Tenant Guide**

As a tenant you have your own portal, separate from the management views above. You can only ever see your own lease, payments, and maintenance requests.

### **B.3.1 Your Dashboard**

After logging in you land on your dashboard, showing your unit, monthly rent, current balance, recent payments, and your maintenance requests.

### **B.3.2 Pay Rent Online**

- From the sidebar, choose "Pay Rent".
- Confirm the amount shown (your current balance) and click "Pay Now".
- You will receive an emailed receipt once the payment is confirmed, and your balance updates immediately.

### **B.3.3 View Your Payment History**

Choose "Payment History" to see every payment you have made against your lease, with its date, amount, method, and receipt number.

### B.3.4 Submit a Maintenance Request

- Choose "Submit Request" from the sidebar or your dashboard.
- Enter a title, choose a category and severity, and describe the issue.
- Submit — you will receive a ticket number by email, and your property manager is notified immediately (with an SMS alert too, if you marked the issue as Emergency).
- You may have at most 5 open requests at a time.

### B.3.5 Respond to a Completed Request

When your maintenance request is marked Completed, open it from your dashboard. You may either Confirm Resolution (optionally rating the work 1–5 stars), which closes the ticket, or Reopen Request with a reason if the issue is not actually fixed, which sends it back to your property manager for reassignment.

## B.4 Maintenance Staff Guide

As Maintenance Staff you see only your own work queue — requests assigned specifically to you.

- Open Maintenance to see your assigned requests.
- Open a request and choose "Mark In Progress" once you begin work.
- When finished, add work notes describing what was done and choose "Mark Completed".
- You cannot see any other role's data — properties, tenants, rent, and reports are not part of your view.

## B.5 Common Questions

| Question                                                        | Answer                                                                                                                                                                                    |
|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why can't I create a new property as a Manager?                 | Property creation is Owner-only by design (Section 2.2) — ask the property owner to add it, then it will be available to you.                                                             |
| Why is a unit still showing as Vacant after I invited a tenant? | The unit only becomes Occupied once a lease document is uploaded and activated (Section B.2.4) — an accepted invitation alone creates the tenant's account but not yet a lease.           |
| Can I edit a payment I recorded by mistake?                     | No — payment records are permanent by design (Section 5.6). Record a correcting entry instead and note the reason.                                                                        |
| Why did my maintenance request get reassigned automatically?    | It didn't — but if its target completion date passes without being marked Completed, the system flags it as overdue and alerts the property manager (UC-23); nothing changes on your end. |

## Appendix C: Updates Since Deliverable 4 (October 2026)

This section brings the document up to date. The rest of the document describes RentalPro as it stood at Deliverable 4, and its figures are deliberately left as they were at that time. Since then, Deliverable 5 closed eleven gaps against the SRS, and the system has gained a billing engine, a property map, a redesigned interface, English, Shona and Arabic interface languages, Harare demonstration data, a tenant history page for each unit and a maintenance summary report, full Arabic and production hardening, and finally deployment to Render. Each change was made additively, behind the existing test suite, with new tests of its own.

### C.1 Summary of Changes

| Area               | What changed                                                                                                                                                                                                                                                                                                                          | New tests |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Deliverable 5      | Eleven gaps against the SRS closed: editing units; property, unit and maintenance photos; tenant ID documents; lease document download; tenant blacklisting; administrator user and role management; staff account creation; configurable late fees; CSV and Excel exports.                                                           | 31        |
| Billing engine     | Charges and sundries, electricity metering, pro rata first month, variable/escalating rent, automated recurring billing (a fifth scheduler job), statements by period, PDF payment receipts, and property expenses with net income. Five new tables: lease_charges, meter_readings, rent_revisions, rent_invoices, property_expenses. | 79        |
| Property map       | Map coordinates on each property, set by clicking a map on the Add/Edit forms; a pin on the property page and a portfolio map at /properties/map. Leaflet is bundled with the app, not loaded from a CDN.                                                                                                                             | 8         |
| Interface redesign | The sidebar was replaced by a horizontal top navigation bar. The login page is a full-screen photograph of the Harare skyline, and each section of the app has its own Harare photograph behind it, with solid white cards and white page titles.                                                                                     | —         |
| Languages          | A language menu in the top bar and on the login page shows the current language and opens a list of English, ChiShona and Arabic; a small translator (backend/i18n.py) with an English fallback; Arabic pages switch to a right-to-left layout; the choice survives logging in and out. Names and places are never translated.        | 14        |
| Tenant unit labels | The tenant dashboard and Pay Rent page show “Avondale Heights · Unit 101” instead of the internal unit code.                                                                                                                                                                                                                          | 1         |
| Logged-in redirect | Opening the home page, the login page, a reset link or an invitation link while signed in now goes to the user's own home page instead of an empty page.                                                                                                                                                                              | 4         |
| Harare demo data   | Five properties in Avondale, the CBD, Eastlea and Vainona with map coordinates; 16 units; nine tenants with Zimbabwean names, national IDs and +263 phone numbers; payment histories with three tenants in arrears; the administrator account Abdur-Rahmaan Ali.                                                                      | —         |
| History & summary  | A History page on every unit lists each tenant who has occupied it, newest first (FR-015); the Reports page has a Maintenance Summary of open, in-progress and closed requests per property for a chosen period (FR-036).                                                                                                             | 13        |

| Area                    | What changed                                                                                                                                                                                                                                                                                                                                                                                        | New tests |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Full Arabic & hardening | Arabic on every page, message, email, SMS and PDF; each user's language saved; every amount with its currency; security headers and a required secret key; Alembic database migrations; optional S3-compatible file storage.                                                                                                                                                                        | 54        |
| Cloud deployment        | render.yaml deploys the Docker image and a PostgreSQL database to Render, the cloud host planned in Deliverable 3, served over HTTPS. Behind the host's proxy the app recognises HTTPS and each visitor's own address (TRUST_FORWARDED_PROTO and CLIENT_IP_HEADER, both off by default), database URLs name the installed PostgreSQL driver, and only one Gunicorn worker runs the background jobs. | 7         |

## C.2 Current Figures

| Figure                    | At Deliverable 4 | Now (October 2026)         |
|---------------------------|------------------|----------------------------|
| Database tables           | 13               | 18                         |
| Jinja2 templates          | 31               | 40                         |
| Route blueprints          | 9                | 10                         |
| Background scheduler jobs | 4                | 5                          |
| Test modules              | 5                | 22                         |
| Automated tests           | 93               | 305                        |
| Statement coverage        | 77%              | 86%                        |
| Interface languages       | 1 (English)      | 3 (English, Shona, Arabic) |

All 305 tests pass with no failures, and the suite still runs entirely offline, with no paid external account needed.

## C.3 Language Support and the SRS

The SRS described multi-language support as Arabic and English. Both are now implemented, together with Shona. Shona, the most widely spoken language in Harare, serves the landlords, managers and tenants the system was built for; Arabic, the second language the SRS named, serves Arabic-speaking residents and property owners in Zimbabwe. Each language is only another phrase table, so a further language, such as Ndebele, can be added the same way.

- **How it works:** Templates wrap interface text in \_("..."); backend/i18n.py looks each phrase up in the table for the chosen language — the Arabic table covers all 613 phrases the app shows or sends, the Shona table the 98 on the main screens — and falls back to English for anything not yet translated, so the app works at every stage of translation.
- **Right-to-left layout:** Arabic is written from right to left, so an Arabic page is marked dir="rtl" and the browser mirrors the layout: the navigation starts from the right and table columns read from right to left (Figure 6), and on the login page the RentalPro name and the language menu swap corners. The stylesheet aligns text to the start of the line rather than to a fixed left or right, so one stylesheet serves both directions. English and Shona stay left to right.

- **Choosing a language:** A language button in the top bar and on the login page shows the current language; clicking its arrow opens a vertical list of English, ChiShona and Arabic, each written in its own language. The list closes on a click elsewhere or the Escape key. The choice survives logging in and out and is saved to the user's account, so it follows them to any browser and their emails and SMS are written in it; the menu's return address is checked so it cannot redirect off-site.
- **Translated:** Arabic covers every page, form option and message, every email and SMS, the PDFs (with joined, right-to-left letters) and the Excel and CSV exports; a test fails if any text the app shows or sends has no Arabic. Shona covers navigation for every role, login and password reset, the dashboards, the main list pages, the tenant portal and status badges; other Shona text falls back to English.
- **Never translated:** Property names, addresses, suburbs, cities and people's names are data, not interface text, and read the same in every language; a test fails if a Harare place name is altered in Shona or Arabic, or given an entry in either phrase table.
- **Tests:** Fourteen tests cover the languages: switching to Shona and to Arabic, the right-to-left direction for Arabic only, the choice surviving login, the tenant portal in Shona and in Arabic, the language menu, unknown language codes, off-site redirects, placeholders kept in every translation, Arabic covering every Shona phrase, and place names. Later tests check that every phrase the app shows or sends has Arabic, and that emails, SMS, PDFs, Excel and CSV follow the reader's language.
- **Caveat:** The Shona and Arabic wording has not yet been reviewed by native speakers.

## C.4 The Current Interface

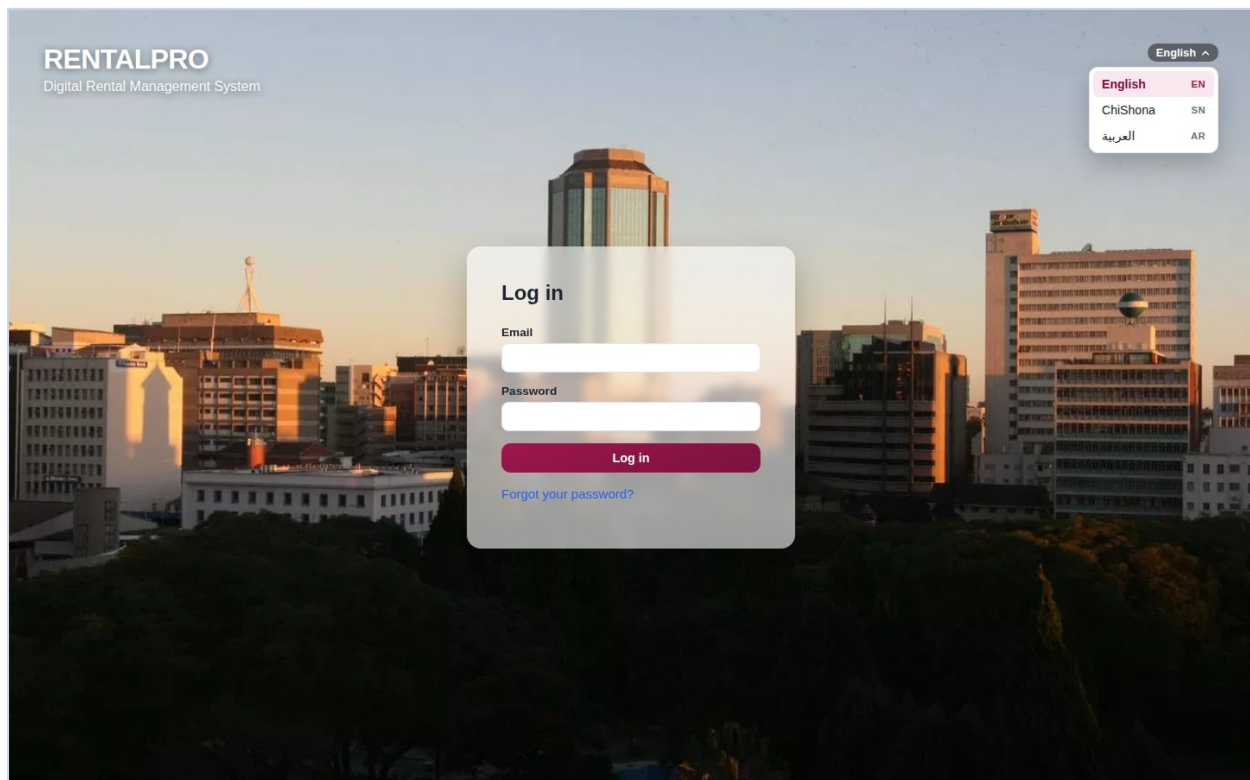


Figure 4: The login page — Harare skyline, translucent card and the language menu, opened

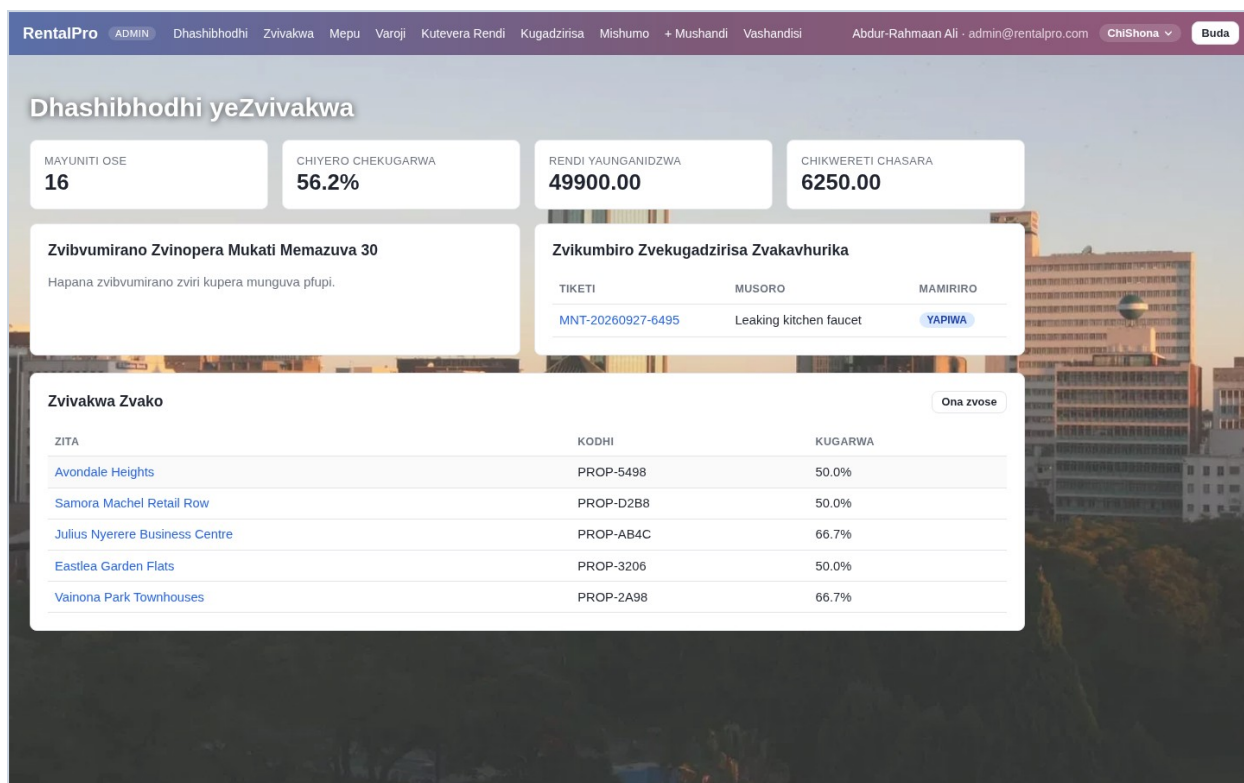


Figure 5: The portfolio dashboard in Shona — names and places stay as entered

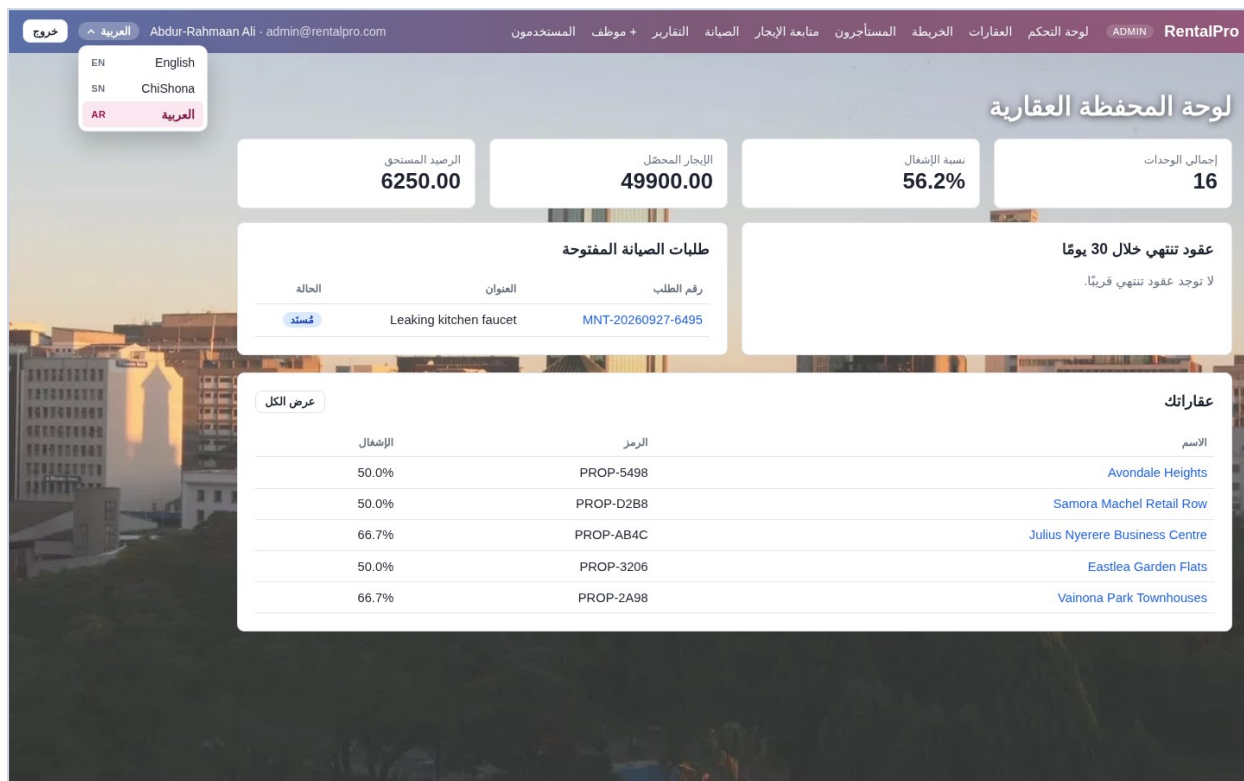


Figure 6: The portfolio dashboard in Arabic — the layout reads right to left and the language menu is open

Choosing a language (addition to Appendix B): click the language button at the top right of any page or of the login page (top left when the page is in Arabic), then pick English, ChiShona or Arabic from the list. The choice stays in place until you change it again.

Managing users (addition to Appendix B): the administrator (admin@rentalpro.com in the demo data) chooses Manage Users in the top bar to see every account with its role and status. To change a role, or to deactivate an account so it can no longer sign in, change the Role or untick Active on that row and choose Save; the change is recorded in the audit log and the user is emailed. + Add Staff creates a Maintenance Staff account and emails the new staff member a link to set a password. The system refuses any change that would leave no active Property Owner.

## C.5 Updated Demo Accounts and Data

All demo accounts use the password demo123. The seed data only loads into an empty database; delete rentalpro.db (or the Postgres volume) to reload it.

| Email                                      | Name                            | Role                                |
|--------------------------------------------|---------------------------------|-------------------------------------|
| admin@rentalpro.com                        | Abdur-Rahmaan Ali               | Administrator                       |
| owner@rentalpro.com                        | Tendai Moyo                     | Property Owner                      |
| manager@rentalpro.com                      | Rutendo Chikore                 | Property Manager                    |
| staff@rentalpro.com                        | Farai Ncube                     | Maintenance Staff                   |
| tenant1@email.com                          | Chipo Mutasa                    | Tenant (Avondale Heights, Unit 101) |
| tenant2@email.com ...<br>tenant9@email.com | Eight further Harare<br>tenants | Tenants across all five properties  |